

MixKVQ: Query-Aware Mixed-Precision KV Cache Quantization for Long-Context Reasoning

Tao Zhang¹, Ziqian Zeng^{1*}, Hao Peng², Huiping Zhuang¹, Cen Chen^{1,3}

¹South China University of Technology, China,

²Beihang University, China,

³Pazhou Laboratory, China,

Correspondence: zqzeng@scut.edu.cn

Abstract

Long Chain-of-Thought (CoT) reasoning has significantly advanced the capabilities of Large Language Models (LLMs), but this progress is accompanied by substantial memory and latency overhead from the extensive Key-Value (KV) cache. Although KV cache quantization is a promising compression technique, existing low-bit quantization methods often exhibit severe performance degradation on complex reasoning tasks. Fixed-precision quantization struggles to handle outlier channels in the key cache, while current mixed-precision strategies fail to accurately identify components requiring high-precision representation. We find that an effective low-bit KV cache quantization strategy must consider two factors: a key channel’s intrinsic quantization difficulty and its relevance to the query. Based on this insight, we propose **MixKVQ**, a novel plug-and-play method that introduces a lightweight, query-aware algorithm to identify and preserve critical key channels that need higher precision, while applying per-token quantization for value cache. Experiments on complex reasoning datasets demonstrate that our approach significantly outperforms existing low-bit methods, achieving performance comparable to a full-precision baseline at a substantially reduced memory footprint.

1 Introduction

Recent pioneering Large Language Models (LLMs), such as OpenAI-o1 (OpenAI, 2024), DeepSeek-R1 (DeepSeek-AI et al., 2025), and Gemini 2.5 Pro (Reid et al., 2024), have demonstrated remarkable reasoning abilities by generating long Chain-of-Thoughts (CoT). To achieve superior performance, these models are trained to generate up to 128K tokens with multiple complex rationales from different perspectives. However, the auto-regressive nature of LLM inference introduces

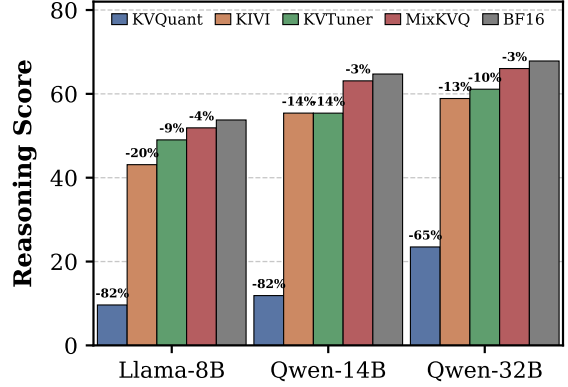


Figure 1: Complex reasoning performance of different 2-bit KV cache quantization. Reasoning score is the average accuracy of AIME 2024-2025, MATH 500, GPQA, and LiveCodeBench.

substantial memory overhead, stemming from the storage demands of the progressively growing Key-Value (KV) cache (Pope et al., 2023). As generating tokens sequentially, LLMs must retain all previous key and value activations in memory to compute attention scores. Consequently, the memory footprint of KV cache grows linearly with the sequence length. For instance, a 32B Qwen2.5 model (Yang et al., 2024), with a batch size of 64 and a sequence length of 32,768 tokens (both pre-filled and decoded), requires approximately 512 GB of GPU memory. This requirement is 8.59× greater than the memory used to store the model’s weights. Furthermore, the attention mechanism must repeatedly access this expanding KV cache at each decoding step, creating significant memory bandwidth pressure. This recurrent data movement leads to bandwidth saturation, establishing a critical memory-bound bottleneck (Yu et al., 2022; Yuan et al., 2024) that severely constrains inference throughput.

More recently, KV cache quantization has emerged as a promising solution. It converts the cache from high-precision floating-point formats to low bit-width integer representations. Channel-

*Corresponding author

wise Key and token-wise Value asymmetric quantization has garnered much attention for its high accuracy and tuning-free nature (Liu et al., 2024; Hooper et al., 2024; Liu et al., 2025a).

However, as demonstrated in Figure 1, existing quantization methods lead to dramatic degradation on complex reasoning tasks when pushed to extreme bit-widths such as 2-bit. Their failures stem from following primary limitations. First, fixed-precision methods (Hooper et al., 2024; Liu et al., 2024; Duanmu et al., 2024; Su et al., 2025a) apply a uniform bit-width across the entire KV cache. This approach fails to accommodate channels with extreme outliers, consequently leading to significant accuracy degradation. Second, existing mixed-precision methods (Li et al., 2025; Chen et al., 2024; Liu et al., 2025b) typically allocate bit-widths based on quantization errors in the key or value caches. Under strict memory constraints, such heuristics allocate higher bit-widths to KV cache segments with larger quantization errors but over-compress those with smaller ones. Yet, these low-error segments may still demand higher precision, resulting in significant degradation on complex reasoning tasks.

The goal of KV cache quantization is a dual objective: it must drastically reduce memory usage without compromising the fidelity of attention computations. However, existing methods often treat minimizing quantization error of key and value cache as a direct proxy for preserving fidelity. This strategy is flawed, as it may allocate higher bit-width to key channels that exhibit large numerical errors but have a negligible impact on the attention computation. Such misallocation yields negligible gains in attention fidelity while inefficiently consuming the limited bit-budget. Our analysis shows that a key channel with large-magnitude activations may not be influential if its corresponding query activations are small. Preserving such channels offers diminishing returns and is an inefficient use of the limited bit budget. Therefore, an effective quantization strategy must consider two factors: a channel’s intrinsic quantization difficulty and its relevance to the query.

Based on this insight, we propose **MixKVQ**, a novel plug-and-play method that introduces a lightweight, hybrid query-aware heuristic to identify and preserve critical channels in higher precision. This heuristic dynamically estimates the potential fidelity impact of quantizing each key channel by combining its intrinsic quantization

difficulty with its relevance to the query. Channels with a high estimated impact are preserved in higher precision. MixKVQ quantizes key cache per-channel with mixed precision and quantizes value cache per-token. To evaluate the effectiveness of our method, we conduct experiments on various models including Llama-3.1 (Dubey et al., 2024), and Qwen2.5 (Yang et al., 2024) family. Compared with the previous quantization method, our approach can achieve superior performance under different average bit widths in mathematical reasoning tasks (MAA, 2024, 2025; Lightman et al., 2024; Rein et al., 2024), as shown in Figure 1. Our contributions are summarized as follows:

- We find that for effective low-bit KV cache quantization, the precision allocated to a key channel must be determined by two factors: its intrinsic quantization difficulty and its dynamic relevance to the query.
- Building on this insight, we propose **MixKVQ**, a novel plug-and-play method for extreme low-bit KV cache quantization. MixKVQ quantizes key cache per-channel with mixed precision and quantizes value cache per-token.
- Experiments on mathematical reasoning benchmarks demonstrate that MixKVQ consistently outperforms existing methods across various compression rates, achieving performance on par with the full-precision baseline.

2 Related Work

KV cache quantization. Recent advances in KV cache quantization focus on low-bit compression and asymmetry-aware optimization. Representative techniques include rotation-based transformations to handle outliers (Ashkboos et al., 2024; Su et al., 2025b; Duanmu et al., 2024), tuning-free asymmetric bit allocation (Liu et al., 2024; Su et al., 2025a; Qian et al., 2025), and mixed-precision adaptation (Li et al., 2025; Hooper et al., 2024). However, most existing quantization methods are proposed for non-reasoning task. Intuitively, KV cache quantization may pose greater risks in this context, as reasoning models typically involve long Chain-of-Thought (CoT) outputs (Wei et al., 2022), which are more prone to quantization error accumulation over the sequence.

KV cache eviction. Researchers have proposed other several KV cache compression techniques.

One direction is KV cache eviction methods (Xiao et al., 2024; Zhang et al., 2023; Han et al., 2024; Cai et al., 2024; Wan et al., 2025; Feng et al., 2025; Tian et al., 2025; Qin et al., 2025) improve computational efficiency during model inference by retaining critical KV entries and discarding non-essential ones. However, eviction operations can lead to irreversible information loss, and some methods rely on predefined strategies, struggling to fully capture the dynamic characteristics of attention mechanisms.

KV cache management. Another strategy is KV cache management, which is an optimization technique to accelerate LLM inference by improving memory utilization. KV cache management methods mainly include prefilling-decoding disaggregation (Qin et al., 2024), low-rank decomposition (Chang et al., 2024; Zhang et al., 2024; Saxena et al., 2024; Chang et al., 2025), offloading (Lee et al., 2024b; Ren et al., 2025), prefetching (Yüzügüler et al., 2025; Dong et al., 2025), and retrieval (Tang et al., 2024; Di et al., 2025). It is worth noting that our fine-grained numerical compression is orthogonal to these methods. For instance, it can be combined with structural compression from low-rank methods, or applied within active pages managed by retrieval systems to optimize data representation.

3 Preliminaries

3.1 Transformer and the KV Cache

The core of a Transformer layer is the self-attention mechanism. The output $\mathbf{o}_i^l$ for a query vector $\mathbf{Q}_i^l$ at layer l is computed by attending to a sequence of key and value vectors:

$$\mathbf{a}_i^l = \text{softmax} \left(\frac{\mathbf{Q}_i^l (\mathbf{K}^l)^\top}{\sqrt{D}} \right), \quad \mathbf{o}_i^l = \mathbf{a}_i^l \mathbf{V}^l, \quad (1)$$

where D is the model hidden size, $\mathbf{a}_i^l$ denotes the attention weight vector for the i -th token at layer l . During autoregressive generation, the matrices $\mathbf{K}^l$ and $\mathbf{V}^l$ contain the key and value vectors from all previous timesteps. To avoid costly recomputation, these matrices are stored in a **KV cache**, which grows linearly with the sequence length.

3.2 KV Cache Quantization

The memory footprint of the KV cache can become an inference bottleneck. Quantization addresses this by storing the cache in a low-bit format. We employ a B -bit asymmetric quantization scheme,

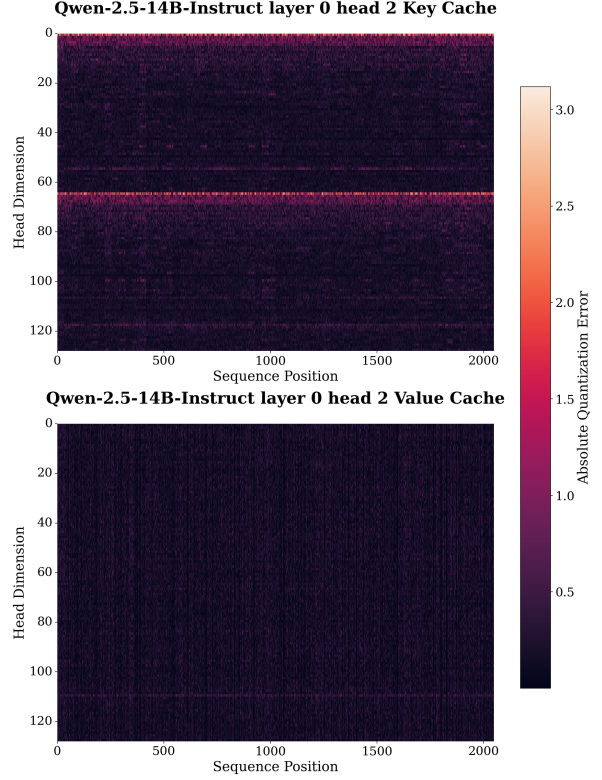


Figure 2: Absolute quantization error of key and value cache for Qwen-2.5-14B-Instruct model.

which approximates a tensor $\mathbf{X}$ with its dequantized representation $\tilde{\mathbf{X}}$:

$$Q(\mathbf{X}) = \text{round} \left(\frac{\mathbf{X} - \mathbf{z}}{s} \right), \quad (2)$$

$$\tilde{\mathbf{X}} = Q(\mathbf{X}) \cdot s + \mathbf{z}. \quad (3)$$

The parameters are a zero-point $\mathbf{z} = \min(\mathbf{X})$ and a scaling factor $s = (\max(\mathbf{X}) - \min(\mathbf{X})) / (2^B - 1)$. The scaling factor s , which is determined by the dynamic range of the tensor, serves as a direct proxy for its quantization sensitivity. A larger s implies that a wider value distribution is compressed into the 2^B discrete levels, leading to coarser granularity. Specifically, the rounding operation in Eq. 2 introduces a quantization error $|x_i - \tilde{x}_i|$ for each $x_i \in \mathbf{X}$. This error is mathematically bounded by $|x_i - \tilde{x}_i| \leq s/2$ (see proof in Appendix A). A single outlier can inflate s , significantly increasing this error bound and degrading the approximation quality for all other elements.

4 Method

In long-context or batched inferences, the primary bottlenecks in memory and speed arise from storing and loading the KV cache (Yuan et al., 2024). An effective way to alleviate this problem is to

Table 1: The error accumulation caused by low-bit KV cache quantization potentially leads to wrong responses in mathematical reasoning tasks.

<i>Question: Let $\triangle ABC$ have circumcenter O and incenter I with $\overline{IA} \perp \overline{OI}$, circumradius 13, and inradius 6. Find $AB \cdot AC$.</i>	
BF16	Given triangle $\triangle ABC$ with circumcenter O and incenter I , where $\overline{IA} \perp \overline{OI}$, circumradius $R = 13$, and inradius $r = 6$. We need to find $AB \cdot AC$ Thus, the product $AB \cdot AC$ is 468.
MixKVQ	Given triangle $\triangle ABC$ with circumcenter O and incenter I , where $\overline{IA} \perp \overline{OI}$, circumradius $R = 13$, and inradius $r = 6$. We need to find $AB \cdot AC$ Thus, the product $AB \cdot AC$ is 468.
KIVI-4bit	Given triangle $\triangle ABC$ with circumcenter O and incenter I , where $\overline{IA} \perp \overline{OI}$, circumradius 13, and inradius 6. We need to find $AB \cdot AC$ So, $480 = x^2 - 2y - (14/13)y = x^2 - (2 + 14/13)y = x^2 - (40/13)y$ Thus, the product $AB \cdot AC$ is 468.
KIVI-2bit	Given triangle $\triangle ABC$ with circumcenter O and incenter I , where $\overline{IA} \perp \overline{OI}$, circumradius $R = 13$, and inradius $r = 6$. We need to find the product $AB \cdot AC$ So, $480 = x^2 - 2y - (14/13)y = x^2 - (2 + 14/13)y = x^2 - (30/13)y$ Thus, the product $AB \cdot AC$ is 429.
KVTuner	Given triangle $\triangle ABC$ with circumcenter O and incenter I , where $\overline{IA} \perp \overline{OI}$, circumradius $R = 13$, and inradius $r = 6$. We need to find the product $AB \cdot AC$ So, $x^2 - (2 + 14/13)y = 480$. Which is $x^2 - (30/13)y = 480$ Thus, the product $AB \cdot AC$ is 429.

Table 2: The results of simulated KV cache quantization with various configurations. All quantization methods are applied at 2-bit precision.

Model	Method	Perplexity ↓	
		WikiText2	C4
Qwen2.5-7B-Instruct	BF16	6.46	2.51
	KIVI-KV4	6.75	2.74
	KIVI-K4V2	6.81	2.77
	KIVI-K2V4	8.13	3.15
	KIVI-KV2	8.87	3.24

reduce the total number of bytes occupied by the KV cache, specifically through quantization. However, existing quantization methods leads to dramatic degradation on complex reasoning tasks when pushed to extreme bit-width, detailed in Section 4.1. To address this, we introduce MixKVQ, a novel method that introduces a lightweight, hybrid query-aware heuristic to preserve critical channels with higher precision, detailed in Section 4.2.

4.1 Quantization Error Analysis

In autoregressive LLMs, KV cache quantization error accumulates across two dimensions: model depth (layer-to-layer) and sequence length (token-to-token). While the quantization error for a single token or layer may be negligible, its cumulative effect across thousands of tokens can be substantial (Li et al., 2025). This leads to phenomena like token flipping and cascading generation errors, analogous to issues in model weight quantization (Lee et al., 2024a). As shown in Table 1, this error

accumulation is particularly detrimental in mathematical reasoning task, where the corruption of a single critical value can invalidate an entire logical chain, squandering significant computational resources.

Key Cache is Generally More Important. Figure 2 visualizes the absolute quantization error in 2 bit of the key and value cache for the Qwen-2.5-14B-Instruct model (layer 0, head 2). Notably, certain channels in the key cache exhibit significantly larger quantization errors. In contrast, the error distribution for the value cache is more uniform and lacks distinct outliers. As shown in Table 2, our experimental results confirm that preserving higher precision for the key cache is more critical for maintaining model performance. This finding aligns with prior work (Liu et al., 2024; Adnan et al., 2024; Li et al., 2025). Therefore, the core challenge is to develop a precision allocation strategy for the key cache that minimizes attention score loss under a constrained memory budget.

Fixed-Precision Methods Struggle with Outliers. Existing fixed-precision methods (Liu et al., 2024; Hooper et al., 2024; Duanmu et al., 2024), perform poorly at low bit-widths like 2-bit. As illustrated in Figure 2, while these methods can adequately compress the value cache, they struggle to handle key cache channels with significant outliers. This results in large quantization errors and critical failures on reasoning tasks.

Suboptimal bit-widths Allocation in Existing Mixed-Precision Methods. Current layer-wise

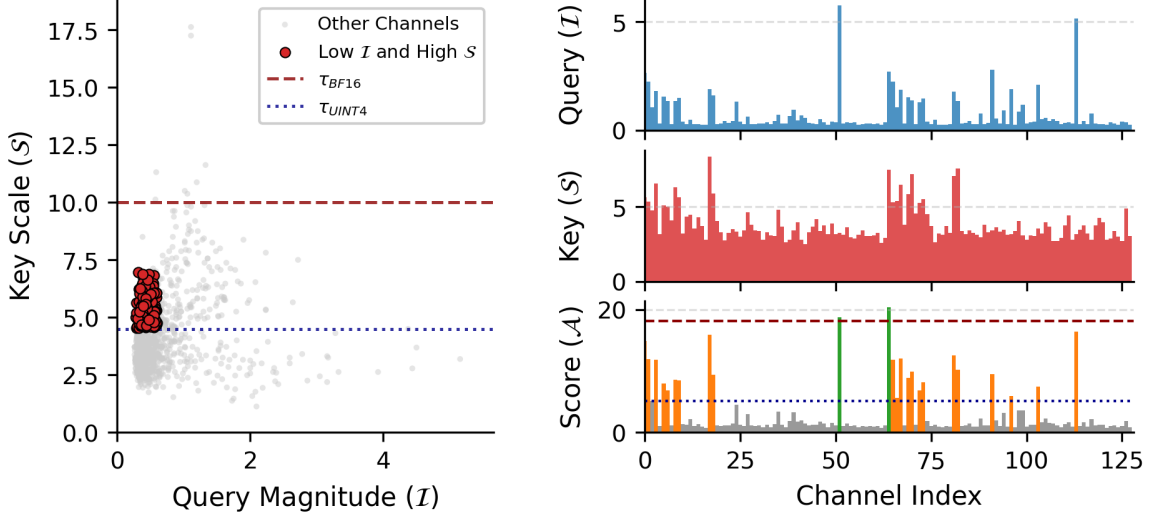


Figure 3: Analysis of Key channel properties on Qwen-2.5-14B-Instruct. Scatter plot of Query magnitude ($\mathcal{I}$) versus Key scale ($\mathcal{S}$) in Layer 0. Here, $\mathcal{I}$ denotes the average activation intensity of the Query vectors and reflects each channel’s contribution to the attention scores. Traditional methods assign high bit-widths to channels with high $\mathcal{S}$ but low $\mathcal{I}$ (shown as blue dots) due to their large $\mathcal{S}$ values; however, these channels are in fact non-crucial for preserving the fidelity of attention score computation. (b) Channel statistics for Head 4. In contrast to the Key scales ($\mathcal{S}$), which are densely clustered and offer limited discriminative capability, the combined salience score ($\mathcal{A} = \mathcal{I} \cdot \mathcal{S}$) effectively isolates critical channels that require high-precision retention. Bar colors denote the adaptive precision levels: green (BF16), orange (INT4), and grey (INT2).

and channel-wise approaches (Li et al., 2025; Chen et al., 2024; Liu et al., 2025b) exhibits shortcomings in bit-widths allocation. Layer-wise methods, due to their static nature, are prone to over-aggressive quantization of sensitive layers (see Appendix B). Channel-wise heuristics assign higher bit-widths to Key channels with larger scales. However, Figure 3(a) shows that the magnitude of Query activations exhibits little correlation with Key scales, with a Pearson correlation coefficient of merely 0.16. Since high Query activation indicates greater importance for preserving attention fidelity, methods that rely solely on scaling factors may preserve Key channels in higher precision (shown as red dots) that are in fact non-critical to accurate attention score computation. Moreover, Figure 3(b) highlights the limited discriminative capability of scales alone: the distribution of $\mathcal{S}$ is highly concentrated, with 80% of values in Head 0 clustered within the narrow range [2.80, 4.46], making it difficult to differentiate truly critical channels. Together, these observations demonstrate that $\mathcal{S}$ alone is insufficient for identifying salient channels.

The goal of KV cache quantization is to reduce the KV cache memory footprint while preserving the fidelity of the attention computation. Our analysis posits that **a key channel with large-magnitude activations may NOT be critical if its**

corresponding query activations are small. Preserving such channels offers diminishing returns and is an inefficient use of the limited bit budget. This motivates the need for a more informative metric that incorporates Query information to enable effective precision allocation, particularly in low-bit regimes.

4.2 MixKVQ

Based on this insight, we propose **MixKVQ**, a novel method that introduces a lightweight query-aware heuristic to identify and preserve critical channels in higher precision. Given that the value cache can be adequately compressed, the main objective is to mitigate the quantization error introduced in the **attention scores**.

Let $\mathbf{Q} \in \mathbb{R}^{L_q \times D}$ and $\mathbf{K} \in \mathbb{R}^{L_k \times D}$ denote the query and key matrices, respectively, where D is the hidden dimension. When $\mathbf{K}$ is quantized to $\tilde{\mathbf{K}}$, the error in the pre-softmax attention scores, denoted as $\mathbf{E}_{attn}$, is defined as:

$$\mathbf{E}_{attn} = \mathbf{Q}(\mathbf{K} - \tilde{\mathbf{K}})^T. \quad (4)$$

Consider the error term for the i -th query token and j -th key token, denoted as $E_{i,j}$. Let $\epsilon_{j,d} = \mathbf{K}_{j,d} - \tilde{\mathbf{K}}_{j,d}$ represent the quantization noise for the key at token j and channel d . The logit error $E_{i,j}$ aggregates the error contributions across the

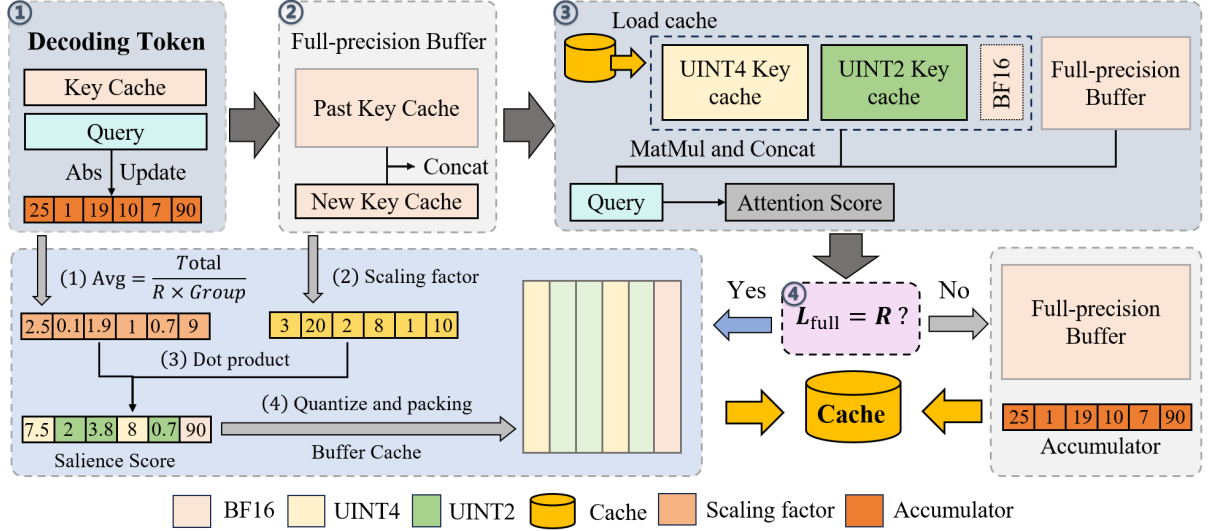


Figure 4: Workflow of MixKVQ. Keys and Values undergo per-channel and per-token quantization, respectively, managed by a full-precision buffer of size R . For Key cache, salience Score $\mathcal{A}_d$ are updated every R tokens within the window, while Value cache utilize uniform 2-bit quantization.

channel dimension:

$$E_{i,j} = \sum_{d=1}^D \mathbf{Q}_{i,d} \cdot \epsilon_{j,d}. \quad (5)$$

To minimize the impact on the attention mechanism, we aim to identify and preserve channels with the highest expected magnitude of error contribution, $\mathbb{E}[|\mathbf{Q}_{i,d} \cdot \epsilon_{j,d}|]$. We approximate this expectation as the product of their individual expected magnitudes: $\mathbb{E}[|\mathbf{Q}_{i,d}|] \cdot \mathbb{E}[|\epsilon_{j,d}|]$. We operationalize this via a heuristic metric, the **Salience Score** ($\mathcal{A}_d$), derived from two low-cost statistics:

Importance Score ($\mathcal{I}_d$): We estimate $\mathbb{E}[|\mathbf{Q}_{i,d}|]$ using the average absolute magnitude of the query channel d over the sequence length L_q :

$$\mathcal{I}_d = \frac{1}{L_q} \sum_{i=1}^{L_q} |\mathbf{Q}_{i,d}|. \quad (6)$$

Sensitivity Score ($\mathcal{S}_d$): For a uniform quantizer with bit-width B , the quantization error $\epsilon_{j,d}$ is bounded by half the scaling factor (i.e., $|\epsilon_{j,d}| \leq \frac{\mathcal{S}_d}{2}$). Thus, We estimate $\mathbb{E}[|\epsilon_{j,d}|]$ using the scaling factor $\mathcal{S}_d$ for channel d :

$$\mathcal{S}_d = \frac{\max(\mathbf{k}_d) - \min(\mathbf{k}_d)}{2^B - 1}, \quad (7)$$

where $\mathbf{k}_d$ represents the vector of key values in channel d across the current tokens.

Finally, the **Salience Score** $\mathcal{A}_d$ is defined as the product of these components:

$$\mathcal{A}_d = \mathcal{I}_d \cdot \mathcal{S}_d. \quad (8)$$

Channels with high $\mathcal{A}_d$ are deemed **critical channels**, as they exhibit both high query relevance and high quantization sensitivity, thereby warranting higher-precision retention.

We designed a three-tiered mixed-precision strategy. While prior studies indicate that 4-bit quantization offers a favorable trade-off between model performance and memory footprint (Liu et al., 2024; Lin et al., 2025; Liu et al., 2025a), we further optimize this balance by allocating bit-widths based on channel salience. To preserve generation quality, the most critical channels are retained in full precision (BF16). Simultaneously, to maximize compression, we quantize moderately critical channels to UINT4 and aggressively compress non-critical channels to UINT2. Formally, we introduce two thresholds, τ_{BF16} and τ_{UINT4} , applied to the salience score $\mathcal{A}_d$:

- **High-Precision (BF16):** Channels with $\mathcal{A}_d > \tau_{\text{BF16}}$ are identified as highly critical and retained in BF16.
- **Medium-Precision (UINT4):** Channels satisfying $\tau_{\text{UINT4}} < \mathcal{A}_d \leq \tau_{\text{BF16}}$ are considered moderately critical and quantized to UINT4.
- **Low-Precision (UINT2):** Channels with $\mathcal{A}_d \leq \tau_{\text{UINT4}}$ are deemed non-critical and compressed to UINT2.

The threshold determination process is detailed in Appendix C.

The overall workflow of MixKVQ is illustrated in Figure 4. We apply per-channel quantization to

Table 3: Performance comparison of BF16, KVquant, KIVI, KVTuner, RotateKV and MixKVQ on AIME 2024-2025, MATH 500, GPQA-Diamond, and LiveCodeBench datasets. The best results are marked in bold.

Model	Method	Bit-width	Accuracy (pass@1)				Avg.
			AIME 2024-2025	MATH 500	GPQA-Diamond	LiveCodeBench	
DeepSeek-R1-Distill-Llama-8B	-	BF16	41.67	89.80	48.99	34.62	53.77
	KIVI	KV4	35.00	86.60	46.46	28.57	49.16
		KV2	31.67	85.80	37.37	17.58	43.11
	KVQuant	KV4	36.67	87.40	44.95	27.47	49.12
		KV2	0	4.20	21.72	12.64	9.64
	RotateKV	KV4	36.67	88.00	42.93	30.22	49.45
	KVTuner-C3.25		30.00	87.80	46.96	31.32	49.02
	MixKVQ-C2.7		40.00	88.20	47.47	31.87	51.89
DeepSeek-R1-Distill-Qwen-14B	-	BF16	61.67	93.60	58.59	45.05	64.73
	KIVI	KV4	55.00	92.20	56.06	42.31	61.39
		KV2	50.00	87.80	52.53	31.32	55.41
	KVQuant	KV4	53.33	90.40	54.58	41.76	60.02
		KV2	0	11.00	22.22	14.29	11.88
	RotateKV	KV4	56.67	92.40	55.56	42.31	61.73
	KVTuner-C2.90		41.67	89.80	50.00	40.11	55.40
	MixKVQ-C2.3		60.00	92.40	56.57	43.41	63.10
DeepSeek-R1-Distill-Qwen-32B	-	BF16	66.67	94.80	62.63	47.25	67.84
	KIVI	KV4	58.33	93.40	59.09	44.51	63.83
		KV2	51.67	89.80	54.54	39.56	58.89
	KVQuant	KV4	60.00	92.60	58.08	43.96	63.66
		KV2	0	42.80	30.81	20.33	23.48
	RotateKV	KV4	61.67	93.40	60.10	42.86	64.51
	KVTuner-C2.91		51.67	91.40	59.60	41.76	61.11
	MixKVQ-C2.3		65.00	94.00	60.10	45.05	66.04

the Key cache and per-token quantization to the Value cache. A buffer of size R stores tokens in full precision until the buffer size is reached, after which they are group quantized. For the Key cache, the parameters $\mathcal{I}_d$ and $\mathcal{S}_d$ are updated periodically (every R tokens). During this update, $\mathcal{I}_d$ is derived by averaging the absolute magnitudes of Query activations within the current window. Conversely, the Value cache undergoes uniform 2-bit per-token quantization. Specifically, we provide the detail for MixKVQ when calculating the attention output in the prefill and decoding phases in Appendix D.

5 Experiments

5.1 Experimental Setup

Models. We evaluate MixKVQ on four models including on the Deepseek-R1-Distill (DeepSeek-AI et al., 2025) series. We choose Deepseek-R1-Distill-Qwen-14B/32B (Yang et al., 2024) and Deepseek-R1-Distill-LLaMA-8B (Dubey et al., 2024). All experiments are conducted on a single NVIDIA A800 GPU (80GB).

Tasks. We evaluate MixKVQ on four reasoning benchmarks: (1) AIME 2024-2025 (MAA, 2024, 2025) and MATH-500 (Lightman et al., 2024) for mathematical reasoning; (2) GPQA-Diamond (Rein et al., 2024) for graduate-level scientific reasoning; and (3) LiveCodeBench (Jain

et al., 2025) for code generation, specifically using the subset from January 1, 2025 to April 6, 2025.

Baseline. We compare our method with KVquant (Hooper et al., 2024), KIVI (Liu et al., 2024), KVTuner (Li et al., 2025), RotateKV (Su et al., 2025b) and BF16 baseline. For all reasoning evaluations, we employ a sampling temperature of 0.6 and a top- p of 0.95. To ensure a fair comparison, we standardize the group size at $G = 32$ and the residual length at $R = 128$ across all quantization methods, including our own.

5.2 Main Results

Table 3 presents the performance of MixKVQ on the AIME 2024-2025, MATH 500, GPQA-Diamond, and LiveCodeBench datasets. The results demonstrate that MixKVQ consistently outperforms existing methods across all evaluated models. For instance, on Qwen-32B model, MixKVQ attains an average accuracy of 66.04%, closely trailing the BF16 baseline of 67.84%. In contrast, the 4-bit baseline RotateKV achieves an average accuracy of 64.51% (a decrease of 3.33%), while the 2-bit baseline KIVI drops to 58.89% (a significant decline of 8.95%). This indicates that MixKVQ can effectively mitigate the quantization error that typically impairs reasoning capabilities.

Table 4: Performance comparison of BF16, KVQuant, KIVI, SKVQ, RotateKV and our MixKVQ on LongBench datasets. The best results within each model group are marked in bold.

Method	Bit-width	Single-document QA		Summarization		Few-shot Learning			Code		Avg. $\uparrow$
		Qasper	MultiFieldQA	QMSum	MultiNews	TREC	TriviaQA	SAMSum	LCC	RepoBench-P	
Mistral-7B-Instruct-v0.3											
-	BF16	41.58	55.23	25.75	27.82	76.00	88.59	47.35	60.53	61.68	53.84
KVQuant	KV4	40.56	53.41	24.72	26.79	75.00	88.21	46.97	58.08	58.13	52.43
	KV2	37.88	49.27	22.16	25.89	70.00	86.53	45.38	56.42	55.94	49.94
KIVI	KV4	40.87	53.68	23.90	26.94	74.00	88.34	47.54	58.64	60.81	52.75
	KV2	38.56	50.47	22.48	26.65	72.00	87.13	47.18	57.15	58.29	51.10
SKVQ	KV4	40.95	53.36	22.04	26.57	76.00	89.32	47.65	58.85	56.58	52.37
	KV2	32.64	46.57	22.20	26.46	76.00	87.52	46.09	57.90	54.16	49.95
RotateKV	KV4	40.64	53.59	22.90	25.09	76.00	88.16	44.08	49.22	47.92	49.73
	KV2	16.69	27.71	18.27	24.88	42.00	48.08	22.20	28.40	31.46	28.85
MixKVQ C2.7		41.19	54.52	26.23	27.66	76.00	88.59	46.89	60.44	61.60	53.68
Llama-3.1-8B-Instruct											
-	BF16	45.53	56.37	25.36	27.19	72.50	91.65	43.62	65.10	58.65	54.00
KVQuant	KV4	44.03	51.67	25.25	26.87	71.00	91.03	43.61	62.15	55.13	52.30
	KV2	36.51	41.85	22.94	25.82	65.00	86.27	41.94	59.88	50.87	47.90
KIVI	KV4	44.42	52.10	25.28	26.86	72.50	91.42	44.07	62.45	55.04	52.68
	KV2	41.02	45.86	24.57	26.79	71.50	91.14	43.41	60.55	51.64	50.72
SKVQ	KV4	44.21	53.96	25.34	26.79	72.50	91.14	42.38	62.44	54.13	52.54
	KV2	35.28	46.27	23.61	26.47	72.50	91.21	42.96	60.63	50.63	49.95
RotateKV	KV4	44.32	53.92	25.00	27.03	71.50	90.84	43.12	63.88	56.39	52.89
	KV2	10.95	22.64	17.50	20.37	21.00	9.84	8.18	30.29	29.20	18.89
MixKVQ C2.7		45.32	55.40	25.29	27.20	72.50	91.78	43.68	64.96	57.26	53.71

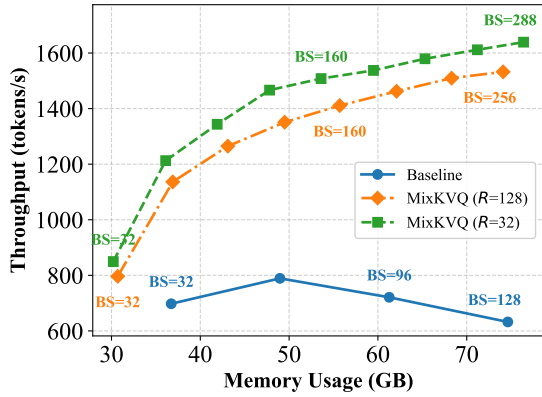


Figure 5: Memory usage and throughput comparison between MixKVQ and 16 bit baseline.

5.3 Long Context Generation Accuracy

Table 4 presents the evaluation on LongBench (Bai et al., 2024) for Mistral-7B and Llama-3.1. Compared to BF16 and competitive baselines, MixKVQ pushes the effective bit-width down to 2.70 bits with negligible performance degradation. This result highlights the efficacy of our mixed-precision strategy, which surpasses quantization baselines in delivering both high fidelity of model performance and memory efficiency.

5.4 Efficiency Comparison

We assess the runtime efficiency of MixKVQ using workloads synthesized from ShareGPT¹, strictly following the vLLM evaluation setting (Kwon

et al., 2023). We push the batch size to memory saturation and compare the throughput and peak memory usage of MixKVQ (residual lengths 32/128) with the FP16 baseline on Llama2-13B-chat. As illustrated in Figure 5, with similar maximum memory usage, MixKVQ enables up to $2.25\times$ larger batch size and gives $2.63\times \sim 2.81\times$ larger throughput. These performance gains are expected to amplify with extended sequence lengths. Furthermore, we plan to integrate MixKVQ into high-performance serving frameworks like vLLM in future work, which we anticipate will unlock substantial additional throughput gains.

Ablation studies investigating the sensitivity to hyperparameters (group size G , residual length R) and the contribution of the query-aware component ($\mathcal{I}_d$) are provided in Appendix E.

6 Conclusion

In this paper, we propose MixKVQ, a plug-and-play KV cache quantization algorithm without the need for any tuning. MixKVQ allocates the precision for a key channel depending on two factors: its intrinsic quantization difficulty and its dynamic relevance to the query. Based on this strategic, MixKVQ quantizes key cache per-channel mixed precision and quantizes value cache per-token. The evaluation shows that MixKVQ achieves optimal performance under extreme low-bit widths in complex reasoning tasks.

¹<https://sharegpt.com/>

Limitations

While MixKVQ achieves substantial KV cache compression via ultra-low-bit quantization, its computational overhead during tensor transformation remains non-negligible despite our GPU-optimized mitigation strategies. This limitation suggests opportunities to enhance runtime efficiency through deeper integration with LLM inference accelerators like vLLM. Additionally, our study does not encompass all attention mechanisms; specifically, we exclude Multi-Head Latent Attention (MLA), which differs significantly from the widely adopted Group-Query Attention (GQA). Furthermore, our latency analysis focuses primarily on memory-bound generation stages, leaving the computational bottlenecks in prompt processing phases insufficiently explored, particularly during batch compression operations across multiple key-value sequences.

References

- Muhammad Adnan, Akhil Arunkumar, Gaurav Jain, Prashant Nair, Ilya Soloveychik, and Purushotham Kamath. 2024. Keyformer: Kv cache reduction through key tokens selection for efficient generative inference. *Proceedings of Machine Learning and Systems*, 7.
- Saleh Ashkboos, Amirkeivan Mohtashami, Maximilian L. Croci, Bo Li, Pashmina Cameron, Martin Jaggi, Dan Alistarh, Torsten Hoefer, and James Hensman. 2024. Quarot: Outlier-free 4-bit inference in rotated llms. *Advances in Neural Information Processing Systems*, 37:100213–100240.
- Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. 2024. LongBench: A bilingual, multi-task benchmark for long context understanding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3119–3137, Bangkok, Thailand. Association for Computational Linguistics.
- Zefan Cai, Yichi Zhang, Bofei Gao, Yuliang Liu, Tianyu Liu, Keming Lu, Wayne Xiong, Yue Dong, Baobao Chang, Junjie Hu, and Wen Xiao. 2024. Pyramidkv: Dynamic kv cache compression based on pyramidal information funneling. *Preprint*, arXiv:2406.02069.
- Chi-Chih Chang, Chien-Yu Lin, Yash Akhauri, Wei-Cheng Lin, Kai-Chiang Wu, Luis Ceze, and Mohamed S. Abdelfattah. 2025. xkv: Cross-layer svd for kv-cache compression. *Preprint*, arXiv:2503.18893.
- Chi-Chih Chang, Wei-Cheng Lin, Chien-Yu Lin, Chong-Yan Chen, Yu-Fang Hu, Pei-Shuo Wang, Ning-Chi Huang, Luis Ceze, Mohamed S. Abdelfattah, and Kai-Chiang Wu. 2024. Palu: Compressing kv-cache with low-rank projection. *Preprint*, arXiv:2407.21118.
- Yidong Chen, Chen Zhang, Rongchao Dong, Haoyuan Zhang, Yonghua Zhang, Zhonghua Lu, and Jidong Zhai. 2024. Mixq: Taming dynamic outliers in mixed-precision quantization by online prediction. In *SC24: International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–15.
- DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Jun-Mei Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiaoling Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, and 179 others. 2025. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *ArXiv*, abs/2501.12948.
- Shangzhe Di, Zhelun Yu, Guanghao Zhang, Haoyuan Li, Tao Zhong, Hao Cheng, Bolin Li, Wanggui He, Fangxun Shu, and Hao Jiang. 2025. Streaming video question-answering with in-context video kv-cache retrieval. *arXiv preprint arXiv:2503.00540*.
- Yanhao Dong, Yubo Miao, Weinan Li, Xiao Zheng, Chao Wang, and Feng Lyu. 2025. Accelerating llm inference throughput via asynchronous kv cache prefetching. *arXiv preprint arXiv:2504.06319*.
- Haojie Duanmu, Zhihang Yuan, Xiuhong Li, Jiangfei Duan, Xingcheng ZHANG, and Dahua Lin. 2024. SKVQ: Sliding-window key and value cache quantization for large language models. In *First Conference on Language Modeling*.
- Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, Anirudh Goyal, Anthony S. Hartshorn, Aobo Yang, Archi Mitra, Archie Sravankumar, Artem Korenev, Arthur Hinsvark, Arun Rao, Aston Zhang, and 510 others. 2024. The llama 3 herd of models. *ArXiv*, abs/2407.21783.
- Yuan Feng, Junlin Lv, Yukun Cao, Xike Xie, and S Kevin Zhou. 2025. Identify critical kv cache in llm inference from an output perturbation perspective. *Preprint*, arXiv:2502.03805.
- Chi Han, Qifan Wang, Hao Peng, Wenhan Xiong, Yu Chen, Heng Ji, and Sinong Wang. 2024. Lm-infinite: Zero-shot extreme length generalization for large language models. *Preprint*, arXiv:2308.16137.
- Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Michael W. Mahoney, Yakun Sophia Shao, Kurt Keutzer, and Amir Gholami. 2024. Kvquant: Towards 10 million context length llm inference with kv cache quantization. In *Advances in Neural Information Processing Systems*, volume 37, pages 1270–1303. Curran Associates, Inc.

- Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. 2025. Live-codebench: Holistic and contamination free evaluation of large language models for code. *The Thirteenth International Conference on Learning Representations*.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. [Efficient memory management for large language model serving with pagedattention](#). In *Proceedings of the 29th Symposium on Operating Systems Principles, SOSP '23*, page 611–626, New York, NY, USA. Association for Computing Machinery.
- Janghwan Lee, Seongmin Park, Sukjin Hong, Minsoo Kim, Du-Seong Chang, and Jungwook Choi. 2024a. [Improving conversational abilities of quantized large language models via direct preference alignment](#). In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 11346–11364, Bangkok, Thailand. Association for Computational Linguistics.
- Wonbeom Lee, Jungi Lee, Junghwan Seo, and Jaewoong Sim. 2024b. [InfiniGen: Efficient generative inference of large language models with dynamic KV cache management](#). In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 155–172, Santa Clara, CA. USENIX Association.
- Xing Li, Zeyu XING, Yiming Li, Linping Qu, Hui-Ling Zhen, Yiwu Yao, Wulong Liu, Sinno Jialin Pan, and Mingxuan Yuan. 2025. Kvtuner: Sensitivity-aware layer-wise mixed-precision kv cache quantization for efficient and nearly lossless llm inference. In *Forty-second International Conference on Machine Learning*.
- Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2024. [Let’s verify step by step](#). In *The Twelfth International Conference on Learning Representations*.
- Yujun Lin, Haotian Tang, Shang Yang, Zhekai Zhang, Guangxuan Xiao, Chuang Gan, and Song Han. 2025. [QServe:w4a8KV4 quantization and system co-design for efficient LLM serving](#). In *Eighth Conference on Machine Learning and Systems*.
- Ruikang Liu, Yuxuan Sun, Manyi Zhang, Haoli Bai, Xianzhi Yu, Tiezheng Yu, Chun Yuan, and Lu Hou. 2025a. Quantization hurts reasoning? an empirical study on quantized reasoning models. *arXiv preprint arXiv:2504.04823*.
- Tengxuan Liu, Shiyao Li, Jiayi Yang, Tianchen Zhao, Feng Zhou, Xiaohui Song, Guohao Dai, Shengen Yan, Huazhong Yang, and Yu Wang. 2025b. [Pm-kvq: Progressive mixed-precision kv cache quantization for long-cot llms](#). *ArXiv*, abs/2505.18610.
- Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. 2024. [KIVI: A tuning-free asymmetric 2bit quantization for KV cache](#). In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 32332–32344. PMLR.
- MAA. 2024. [American invitational mathematics examination - aime](#). In *American Invitational Mathematics Examination - AIME 2024*.
- MAA. 2025. [American invitational mathematics examination - aime](#). In *American Invitational Mathematics Examination - AIME 2025*.
- OpenAI. 2024. [Introducing openai o1](#).
- Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. 2023. [Efficiently scaling transformer inference](#). In *Proceedings of Machine Learning and Systems*, volume 5, pages 606–624. Curran.
- Tao Qian, Yu Wenyuan, and Zhou Jingren. 2025. [Asymkv: Enabling 1-bit quantization of KV cache with layer-wise asymmetric quantization configurations](#). In *Proceedings of the 31st International Conference on Computational Linguistics*, pages 2316–2328, Abu Dhabi, UAE. Association for Computational Linguistics.
- Ruoyu Qin, Zheming Li, Weiran He, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. 2024. [Mooncake: A kvcache-centric disaggregated architecture for llm serving](#). *Preprint*, arXiv:2407.00079.
- Ziran Qin, Yuchen Cao, Mingbao Lin, Wen Hu, Shixuan Fan, Ke Cheng, Weiyao Lin, and Jianguo Li. 2025. [CAKE: Cascading and adaptive KV cache eviction with layer preferences](#). In *The Thirteenth International Conference on Learning Representations*.
- Machel Reid, Nikolay Savinov, Denis Teplyashin, Dmitry Lepikhin, Timothy P. Lillicrap, Jean-Baptiste Alayrac, Radu Soricut, Angeliki Lazaridou, Orhan Firat, Julian Schrittwieser, Ioannis Antonoglou, Rohan Anil, Sebastian Borgeaud, Andrew M. Dai, Katie Millican, Ethan Dyer, Mia Glaese, Thibault Sottiaux, Benjamin Lee, and 654 others. 2024. [Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context](#). *ArXiv*, abs/2403.05530.
- David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R Bowman. 2024. [Gpqa: A graduate-level google-proof q&a benchmark](#). In *First Conference on Language Modeling*.
- Zebin Ren, Krijn Doekemeijer, Tiziano De Matteis, Christian Pinto, Radu Stoica, and Animesh Trivedi. 2025. An i/o characterizing study of offloading llm models and kv caches to nvme ssd. In *Proceedings of the 5th Workshop on Challenges and Opportunities of Efficient and Performant Storage Systems*, pages 23–33.

- Utkarsh Saxena, Gobinda Saha, Sakshi Choudhary, and Kaushik Roy. 2024. [Eigen attention: Attention in low-rank space for kv cache compression](#). *Preprint*, arXiv:2408.05646.
- Yi Su, Yuechi Zhou, Quantong Qiu, Juntao Li, Qingrong Xia, Ping Li, Xinyu Duan, Zhefeng Wang, and Min Zhang. 2025a. [Accurate KV cache quantization with outlier tokens tracing](#). In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 12895–12915, Vienna, Austria. Association for Computational Linguistics.
- Zunhai Su, Zhe Chen, Wang Shen, Hanyu Wei, Linge Li, Huangqi Yu, and Kehong Yuan. 2025b. [Rotatekv: Accurate and robust 2-bit kv cache quantization for llms via outlier-aware adaptive rotations](#). *Preprint*, arXiv:2501.16383.
- Hanlin Tang, Yang Lin, Jing Lin, Qingsen Han, Shikuan Hong, Yiwu Yao, and Gongyi Wang. 2024. Razor: Efficient kv cache compression through retrieval heads. *arXiv preprint arXiv:2407.15891*.
- Yuxuan Tian, Zihan Wang, Yebo Peng, Aomufei Yuan, Zhiming Wang, Bairen Yi, Xin Liu, Yong Cui, and Tong Yang. 2025. [Keepkv: Eliminating output perturbation in kv cache compression for efficient llms inference](#). *Preprint*, arXiv:2504.09936.
- Zhongwei Wan, Xinjian Wu, Yu Zhang, Yi Xin, Chaofan Tao, Zhihong Zhu, Xin Wang, Siqi Luo, Jing Xiong, Longyue Wang, and Mi Zhang. 2025. [D2o: Dynamic discriminative operations for efficient long-context inference of large language models](#). *Preprint*, arXiv:2406.13035.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, brian ichter, Fei Xia, Ed Chi, Quoc V Le, and Denny Zhou. 2022. [Chain-of-thought prompting elicits reasoning in large language models](#). In *Advances in Neural Information Processing Systems*, volume 35, pages 24824–24837. Curran Associates, Inc.
- Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. 2024. [Efficient streaming language models with attention sinks](#). In *The Twelfth International Conference on Learning Representations*.
- Qwen An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Guanting Dong, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxin Yang, Jingren Zhou, Junyang Lin, and 25 others. 2024. [Qwen2.5 technical report](#). *ArXiv*, abs/2412.15115.
- Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. [Orca: A distributed serving system for Transformer-Based generative models](#). In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 521–538, Carlsbad, CA. USENIX Association.
- Zhihang Yuan, Yuzhang Shang, Yang Zhou, Zhen Dong, Chenhao Xue, Bingzhe Wu, Zhikai Li, Qingyi Gu, Yong Jae Lee, Yan Yan, Beidi Chen, Guangyu Sun, and Kurt Keutzer. 2024. [Llm inference unveiled: Survey and roofline model insights](#). *ArXiv*, abs/2402.16363.
- Ahmet Caner Yüzügüler, Jiawei Zhuang, and Lukas Cavigelli. 2025. Preserve: Prefetching model weights and kv-cache in distributed llm serving. *arXiv preprint arXiv:2501.08192*.
- Rongzhi Zhang, Kuang Wang, Liyuan Liu, Shuohang Wang, Hao Cheng, Chao Zhang, and Yelong Shen. 2024. [Lorc: Low-rank compression for llms kv cache with a progressive compression strategy](#). *Preprint*, arXiv:2410.03111.
- Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang "Atlas" Wang, and Beidi Chen. 2023. [H2o: Heavy-hitter oracle for efficient generative inference of large language models](#). In *Advances in Neural Information Processing Systems*, volume 36, pages 34661–34710. Curran Associates, Inc.

A Derivation of Quantization Error Bound

In this section, we provide a formal derivation for the quantization error bound mentioned in Section 3.2 (assuming your main text is in a section labeled like this).

Recall the asymmetric quantization and dequantization operations defined as:

$$q_i = \text{round}\left(\frac{x_i - z}{s}\right), \quad (9)$$

$$\tilde{x}_i = q_i \cdot s + z, \quad (10)$$

where $x_i \in \mathbf{X}$ is the original value, q_i is the quantized integer, $\tilde{x}_i$ is the dequantized approximation, z is the zero-point, and s is the scaling factor ($s > 0$).

We aim to find the upper bound of the absolute quantization error, denoted as $|x_i - \tilde{x}_i|$. By substituting the definition of $\tilde{x}_i$ into the error term, we have:

$$|x_i - \tilde{x}_i| = |x_i - (q_i \cdot s + z)|. \quad (11)$$

Rearranging the terms inside the absolute value operator:

$$|x_i - \tilde{x}_i| = |(x_i - z) - q_i \cdot s|. \quad (12)$$

We can factor out the scaling factor s :

$$|x_i - \tilde{x}_i| = \left| s \cdot \left(\frac{x_i - z}{s} - q_i \right) \right|. \quad (13)$$

Since s is a positive scalar derived from the dynamic range, we can move it outside the absolute value:

$$|x_i - \tilde{x}_i| = s \cdot \left| \frac{x_i - z}{s} - q_i \right|. \quad (14)$$

Let $y_i = \frac{x_i - z}{s}$. By definition, $q_i = \text{round}(y_i)$. The rounding operation $\text{round}(\cdot)$ maps a real number to the nearest integer. A fundamental property of this operation is that the absolute difference between a real number and its nearest integer is at most 0.5:

$$|y_i - \text{round}(y_i)| \leq \frac{1}{2}. \quad (15)$$

Substituting this inequality back into our error equation:

$$\begin{aligned} |x_i - \tilde{x}_i| &= s \cdot |y_i - q_i| \\ &\leq s \cdot \frac{1}{2} \\ &= \frac{s}{2}. \end{aligned} \quad (16)$$

Thus, the quantization error for any element x_i is bounded by half the scaling factor s .

B Failure Case Analysis of KVTuner

KVTuner operates by leveraging calibration data to statically identify entire layers deemed “less critical.” To meet a target memory budget, these designated layers are then subjected to aggressive K2V2 quantization. However, this layer-level approach has a fundamental limitation. As illustrated in Figure 6, even within these so-called non-critical layers, there exist specific dimensions that are difficult to quantize due to outlier values. Uniformly applying an aggressive quantization policy to the entire layer results in significant information loss in these critical dimensions, introducing substantial errors that degrade the model’s performance on complex, multi-step reasoning tasks.

C Threshold Search Strategy

This section details the procedure for determining the importance thresholds, τ_{BF16} and τ_{INT4} , which govern the mixed-precision allocation in the KV cache. We formulate the threshold selection as a dual-objective optimization problem. This framework facilitates the joint optimization of both hyperparameters to effectively navigate the trade-off between memory compression and performance fidelity.

C.1 Optimization Setup

We employ the OPTUNA framework to conduct a joint search for τ_{BF16} and τ_{INT4} within the search space $[0.1, 2.0]$. The optimization aims to satisfy two conflicting objectives simultaneously:

1. **Maximize Accuracy:** We evaluate the quantized model on the GSM8K benchmark. This metric ensures that the quantization process preserves the model’s complex chain-of-thought reasoning abilities.
2. **Minimize Effective Bit-width (B_{eff}):** We aim to minimize the average bit-width per parameter, defined as:

$$B_{\text{eff}} = \frac{1}{N} \sum_{i=1}^N b_i(\tau_{\text{BF16}}, \tau_{\text{INT4}}) \quad (17)$$

where N is the total number of channels, and $b_i \in \{16, 4, 2\}$ denotes the bit-width assigned to the i -th channel based on the selected thresholds.

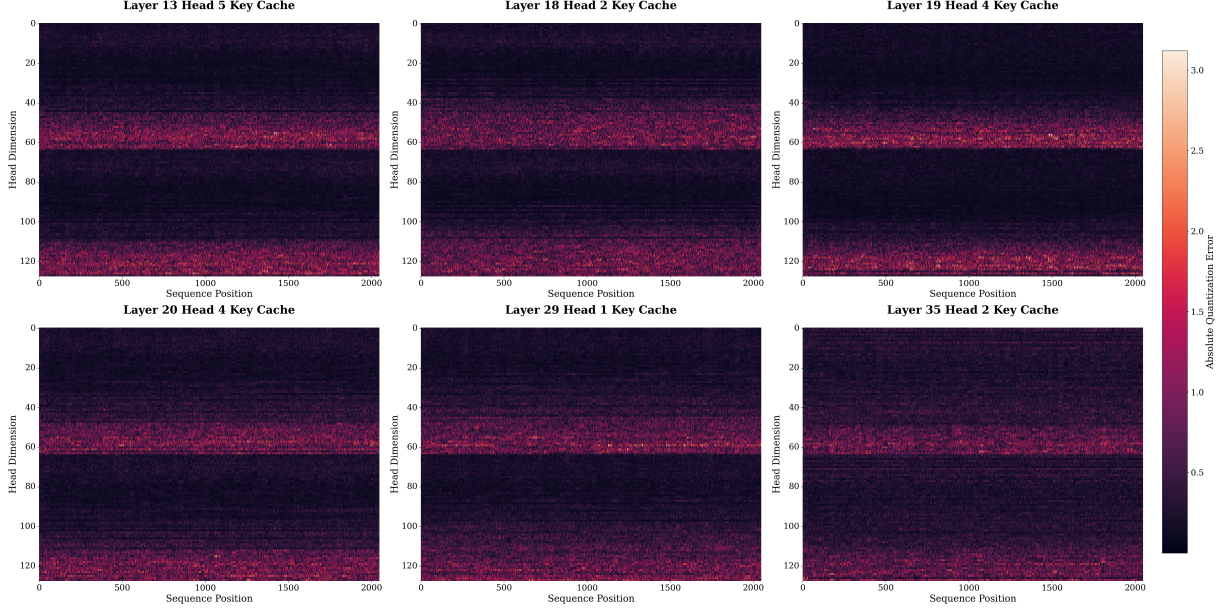


Figure 6: Failure case analysis of the KVTuner method. KVTuner statically identifies many layers as non-critical and applies a uniform, aggressive K2V2 (2-bit key, 2-bit value) quantization policy to them. However, as visualized in the heatmaps, even these targeted layers contain specific dimensions with outlier features that resist effective quantization.

C.2 Selection Process

We utilize the Tree-structured Parzen Estimator (TPE) sampler to efficiently explore the hyperparameter space. By executing 30 trials, we construct a Pareto frontier that visualizes the optimal trade-offs between accuracy and memory efficiency. From this frontier, we select the configuration that yields the highest accuracy while maintaining the effective bit-width below a strict constraint.

As illustrated in Figure 7, the optimal thresholds vary across architectures, reflecting their differing sensitivities to quantization. For the **R1-Llama-8B**, we select thresholds $(\tau_{BF16}, \tau_{INT4}) = (1.44, 0.79)$, resulting in an effective bit-width of 2.7 bits. In contrast, the **R1-Qwen-7B** model exhibits higher sensitivity, requiring more conservative thresholds of $(0.63, 0.41)$ which yield 3.4 bits. Larger models demonstrate greater robustness to compression. Specifically, **R1-Qwen-14B** and **R1-Qwen-32B** both achieve an effective bit-width of 2.3 bits with thresholds of $(1.52, 1.60)$ and $(1.85, 1.58)$, respectively.

D Detailed Implementations

This section details the implementation of the MixKVQ algorithm proposed in Section 4.

To ensure compatibility with modern Large Language Model architectures, our quantization granu-

larity is strictly aligned with the attention mechanism. For models employing Grouped Query Attention (GQA), importance scores are computed at the KV head group level. Specifically, we aggregate query magnitudes from all query heads corresponding to a shared KV head to determine the appropriate quantization precision.

Storage Layout The KV cache is organized into three distinct components to balance memory efficiency and retrieval precision:

- **Quantized Storage ($Q(X_{K/V})$):** This component stores the majority of historical states. Data is packed into low-bit contiguous tensors (e.g., UINT4 or UINT2) to maximize memory throughput.
- **Sparse Outlier Storage ($X_{K_{BF16}}$):** Salient channels identified by the importance thresholds are retained in full BF16 precision. These outliers are managed using sparse formats to minimize storage overhead.
- **High-Precision Residual Buffer (X_R):** A compact buffer maintained in full precision to temporarily store the most recent tokens before quantization.

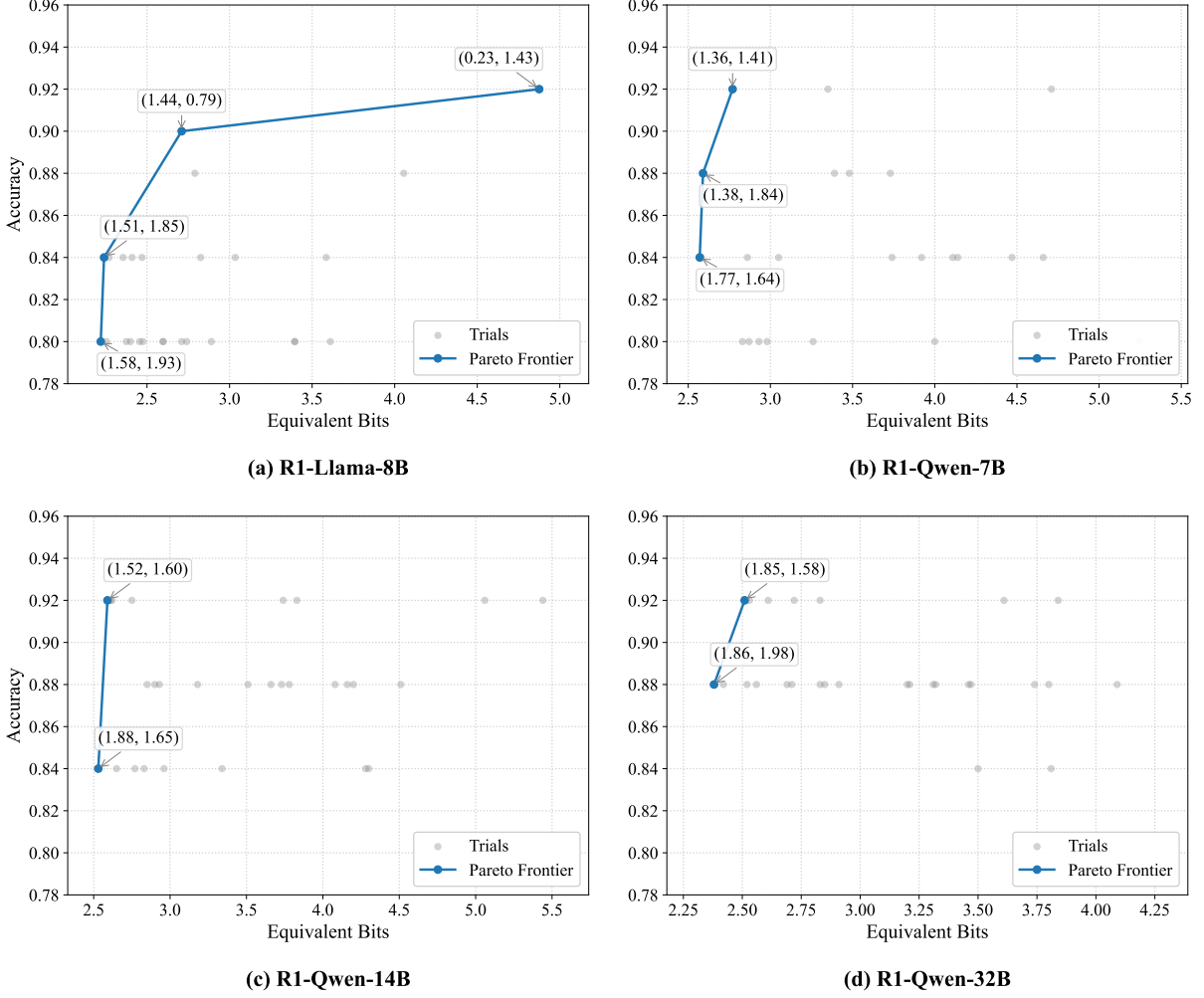


Figure 7: Pareto frontier of different models with the MixKVQ quantization mode on the 25 data slices of the GSM8K dataset.

D.1 Amortized Scheduling via Lazy Updates

To mitigate the computational overhead associated with frequent re-quantization, we employ the lazy update strategy controlled by a residual length hyperparameter R .

During the decoding phase, newly generated Key and Value states are initially appended to the full-precision residual buffer $\mathbf{X}_R$. Consequently, quantization operations are not executed at every generation step. Computationally intensive tasks, including channel selection, outlier extraction, and bit-packing, are triggered exclusively when the buffer length reaches R . Upon triggering, the accumulated block is processed via the KeyQuant procedure, partitioned into quantized and outlier components, and subsequently merged into the main cache. The buffer is then reset. This lazy update mechanism serves a critical dual purpose: it not only amortizes the computational overhead across R decoding steps but also functions as a tempo-

ral stabilization window. Since channel saliency ($\mathcal{I}_d$) often exhibits transient volatility due to local attention patterns on recent tokens, immediate quantization risks assigning precision based on unstable statistics. By isolating these volatile states within $\mathbf{X}_R$, we defer the quantization decision until tokens transition out of the local window.

D.2 Efficient Online Saliency Estimation

Calculating the cumulative importance of KV channels typically necessitates scanning the entire query history, which is computationally prohibitive. To address this, we maintain a running accumulator of query magnitudes within the cache. At each decoding step, the magnitude of the current query token is added to this accumulator.

Furthermore, for architectures incorporating Rotary Positional Embeddings (RoPE), saliency evaluation is performed post-transformation. Specifically, importance scores are calculated after apply-

Table 5: Ablation study of MixKVQ by changing group size G and residual length R .

Model	Group Size	PPL↓
Llama2-13B-chat	32	7.05
	64	7.10
	128	7.12
Model	Residual Length	PPL↓
Llama2-13B-chat	32	7.09
	64	7.04
	96	7.03
	128	7.05
	256	7.03

Table 6: Ablation study of MixKVQ by isolating query-aware component.

Model	Method	AIME 2024-2025
R1-Qwen-14B	error-only	53.33
	MixKVQ	60.00
R1-Llama-8B	error-only	33.33
	MixKVQ	40.00

ing rotary embeddings to the query and key states. This ensures that the metric accurately reflects the attention distribution in the rotated space.

E Ablation Study

Specifically, we investigate the sensitivity of model performance to the quantization group size G and residual length R , and further isolate the contribution of the query-aware saliency term ($\mathcal{I}_d$) against error-only baselines..

The effect of group size. We fix the residual length at 128 and the sink length at 32. We vary the group sizes to 32, 64, and 128. From Table 5, we observe that PPL decreases with group sizes. Note the zero-point and the scaling factor are calculated according to this group size. The choice of group size will greatly impact the KV cache compression effect under a long input.

The effect of residual length. We fix the group size at 32 and the sink length at 32. We vary the residual length across 32, 64, 96, 128, and 256. As shown in Table 5, there is no consistent pattern between residual lengths and model accuracy. Although no significant differences were observed among the tested residual lengths 32, 64, 96, 128, 256, a sufficiently large residual length remains crucial, offering considerable performance boosts for difficult tasks.

Necessity of the query-aware component. To validate the efficacy of our composite metric $\mathcal{A}_d = \mathcal{I}_d \times \mathcal{S}_d$, we conducted a comparative analysis against a Error-only baseline, which determines precision solely based on channel magnitude ($\mathcal{A}_d = \mathcal{S}_d$). We evaluated the variant on the AIME 2024-2025 reasoning benchmark. As presented in Table 6, the full MixKVQ method outperforms the error-only baseline. This result confirms that relying exclusively on quantization error minimization is insufficient, explicitly incorporating query-dependent importance ($\mathcal{I}_d$) is indispensable for preserving model fidelity during complex reasoning tasks.

Table 7: Per-layer time breakdown (%) and call rates across decode steps.

Operation	Time Breakdown (%)	# of Calls (%)
Channel Selection	2.17	3.13
Attention	64.62	100
MLP	33.21	100

F Computation Overhead

In Table 7, we report operation-level breakdowns for R1-Qwen-7B. While Channel Selection comprise 2.17% of per-layer execution, which enables a KV cache memory savings of over 79% (compressing from 16-bit to 3.4-bit).

G Additional Experiments

In this section, we benchmark MixKVQ on DeepSeek-R1-Distill-Qwen-7B (Yang et al., 2024).

Table 8: Performance comparison of BF16, KVquant, KIVI, KVTuner, RotateKV and MixKVQ on AIME 2024-2025, MATH 500 and GPQA-Diamond datasets. The maximum number of generation tokens is limited to 32,768. The best results are marked in bold.

Model	Method	Bit-width	Accuracy (pass@1)				Avg.
			AIME 2024-2025	MATH 500	GPQA-Diamond	LiveCodeBench	
DeepSeek-R1-Distill-Qwen-7B	-	BF16	46.67	89.00	49.49	23.08	52.06
	KIVI	KV4	36.67	87.00	44.94	20.88	47.37
		KV2	30.00	84.60	35.35	16.48	41.61
	KVQuant	KV4	33.33	87.40	41.41	19.44	45.40
		KV2	1.67	37.80	25.76	5.49	17.68
	RotateKV	KV4	33.33	86.40	42.93	18.33	45.25
	KVTuner-C3.92		31.67	86.00	39.90	19.23	44.20
	MixKVQ-C3.4		40.00	88.20	45.45	20.33	48.49